

Main

PRD: Shuyi Website Frontend Revamp — Next.js, Drag-and-Drop CMS, Admin Dashboard, Tarot Web, Membership Optimization

1. Product Name

Shuyi Website Frontend Revamp

2. Updated Product Decision

Shuyi will no longer use Elementor or WordPress as the main frontend website stack.

The website should be rebuilt as a modern, maintainable, API-driven web frontend using Next.js, React, TypeScript, and a reusable component system.

The existing PHP backend should remain as the business logic and API layer for the current phase.

The website should support selected drag-and-drop content editing for marketing and content sections, but core product logic such as tarot, 数易测算, membership entitlement, payment, user records, and authentication should remain controlled by backend APIs and frontend code.

The mobile app remains the main Shuyi product experience. The website should support preview, acquisition, SEO, user education, limited interaction, and conversion into the app.

3. Main Objective

The objective is to rebuild the Shuyi website into a faster, cleaner, more maintainable, mobile-first, SEO-friendly, app-connected frontend.

The new website should support:

Light mode and dark mode

Simple 测算 preview

Tarot lite website experience

Partial 测算记录 preview

- Membership page and upgrade prompts
- Lightweight user dashboard
- Admin dashboard / back office
- Drag-and-drop content editing for marketing sections
- App download and open-app conversion
- Backend API integration
- SEO landing pages
- Analytics tracking
- Reusable frontend components
- Clear separation between frontend display and backend business logic

4. Business Justification

The reason for moving away from Elementor and WordPress is not only to avoid Elementor's yearly fee.

Avoiding Elementor's yearly subscription cost is a supporting reason, but the stronger reason is that Shuyi's website is becoming more interactive and product-driven.

The website now needs tarot preview, calculation flows, membership display, user dashboard, partial records, app deep links, dark mode, analytics, and reusable UI components. These requirements are better handled by a modern frontend stack such as Next.js instead of WordPress and Elementor.

The key business reasons are:

- Better long-term maintainability
- Less plugin dependency
- Better frontend control
- Better performance
- Better mobile responsiveness
- Better API integration
- Better support for app-like website features
- Better ability to match the Flutter app design
- Cleaner future developer handover
- Lower recurring plugin dependency

Elementor fee saving is a bonus. The main reason is scalability, maintainability, performance, and product flexibility.

5. Recommended Tech Stack

5.1 Public Website Frontend

Use:

- Next.js
- React
- TypeScript
- Tailwind CSS
- Reusable component system
- Dark mode support
- SEO metadata support
- Static and dynamic routing
- API-driven frontend architecture

The website frontend should be treated as a standalone web frontend, not as WordPress pages.

5.2 Styling and UI System

Use:

- Tailwind CSS
- Reusable components
- Design tokens
- Consistent spacing system
- Reusable buttons
- Reusable cards
- Reusable form components
- Reusable result components
- Reusable membership cards
- Reusable tarot card components
- Light/dark theme variables

The design system should allow Shuyi to maintain a consistent brand across homepage, tarot, 测算, membership, dashboard, and admin-related pages.

5.3 Backend

Continue using the existing PHP backend for the current phase.

The PHP backend should provide APIs for:

- Tarot lite result generation
- Simple 测算 result generation
- User login or session identification
- Partial 测算记录
- Membership status
- Membership entitlement

- Payment or package confirmation
- User profile
- App deep link generation
- Result saving
- Input validation
- Security checks
- Admin dashboard operations

The frontend should not directly access the database.

5.4 Database

Use the existing database layer managed by the backend.

The website frontend should not directly query the database.

All database-related actions should go through backend APIs.

5.5 Drag-and-Drop Content Management

The website should support drag-and-drop or visual content editing for selected marketing and content areas.

Recommended options:

- Builder.io
- Plasmic
- Directus
- Strapi with structured content blocks
- Custom block-based admin editor

For Shuyi, the recommended practical option is:

Next.js frontend + Builder.io or Plasmic for selected visual editing sections + PHP backend APIs for product logic.

If cost control and full ownership are more important, use a custom block-based admin editor instead of a third-party visual builder.

5.6 Admin Dashboard

The platform should include a separate admin dashboard / back office.

Recommended options:

PHP-based admin dashboard if the backend developer is stronger in PHP
React Admin / Refine / Ant Design Pro if the developer is stronger in React
Directus if a ready-made admin and content management system is preferred
Custom Next.js admin if full control is required

Recommended practical approach:

Use existing PHP backend as the source of truth, expose secure admin APIs, and build either a PHP-based admin panel or React-based admin dashboard.

5.7 Hosting

Recommended hosting:

Frontend: Vercel or similar frontend hosting
Backend: Existing PHP server or existing backend infrastructure
Database: Existing MySQL or current production database
Static assets: CDN or cloud object storage if required

5.8 Analytics

Use:

GA4
Meta Pixel if ads are used
TikTok Pixel if ads are used
Backend event tracking for important product events
Conversion tracking for app download and open-app actions

6. Product Positioning

The website should be positioned as:

A light preview of Shuyi
A trust-building brand website
An SEO acquisition channel
A place for users to try simple 测算
A place for users to try tarot lite
A place for users to understand membership benefits
A limited place to view summary-level records
A conversion funnel into the Shuyi App

The website should not provide the full app experience.

The Shuyi App should remain the place for:

- Full reports
- Detailed tarot readings
- Deep personalized interpretation
- Daily engagement
- Energy points
- Membership benefits
- Full saved history
- Advanced analysis
- Push reminders
- Long-term retention
- Full monetization experience

7. User Types

7.1 New Visitor

A new visitor may arrive from search, social media, ads, or shared links.

They should quickly understand what Shuyi does, try a simple calculation or tarot preview, and be guided toward the app.

7.2 Returning Website User

A returning website user may come back to check a previous light result, try another simple calculation, or revisit membership information.

7.3 Existing App User

An existing app user may visit the website to check content, open a shared link, view partial records, or access membership information.

The website should not confuse them or duplicate the full app experience.

7.4 Admin / Content Editor

A non-technical admin should be able to update selected marketing content, banners, FAQ, SEO content, and landing page sections through drag-and-drop or visual editing.

They should not need to touch code for routine marketing updates.

7.5 Operations Admin

An operations admin should be able to manage users, membership, records, packages, support lookup, and basic platform settings through a secure admin dashboard.

8. Core Website Pages

8.1 Homepage

The homepage should explain what Shuyi does and guide users toward either trying a light calculation, trying tarot lite, or downloading/opening the app.

Homepage sections should include:

- Hero section
- Brand introduction
- Core value proposition
- Popular 测算 entry points
- Tarot lite entry point
- Membership benefit preview
- Sample result preview
- Why use Shuyi
- App benefits
- Trust section
- FAQ preview
- App download CTA
- Footer

8.2 测算 Landing Page

This page should list all available website-level calculations.

Each calculation card should include:

- Name
- Short description
- Estimated time
- Preview-only label
- CTA to start
- CTA to unlock full report in app

8.3 Tarot Lite Page

The Tarot Lite website page should be developed based on the existing Flutter tarot app flow and design reference.

The developer should review the Flutter tarot code and rebuild the website version in Next.js / React.

The website tarot version should include:

- Question input
- Topic selection if applicable
- Card draw animation
- Limited number of cards
- Simplified interpretation
- Preview-only result
- CTA to continue in Shuyi App
- Optional saved preview record

The website tarot should not expose the full tarot app experience unless approved.

8.4 Simple Calculation Result Page

The result page should show:

- Result summary
- Basic explanation
- Limited recommendation
- Preview-only disclaimer
- App CTA
- Save preview option if supported
- Share option if supported

The page should clearly state that the website result is a simplified preview.

Chinese copy:

“此结果为简版预览。如需查看完整个人化解读, 请前往数易 App。”

English copy:

“This is a simplified preview. For a complete personalized interpretation, please continue in the Shuyi App.”

8.5 测算记录 Page

The website may show limited calculation records.

The page should include:

- Recent website records
- Calculation type
- Date created
- Short result summary
- Preview-only label
- CTA to open full record in app

The website should not expose full app-level records unless backend rules allow it.

8.6 Membership Page

Create a clear membership page for website users.

The page should include:

- Free preview benefits
- Elite membership benefits
- VIP membership benefits
- One-off package explanation if applicable
- App-only benefits
- Website-accessible benefits
- CTA to upgrade
- CTA to open app
- FAQ about membership

Membership logic should come from the backend API.

The website should not hardcode membership status or entitlement rules.

8.7 User Profile / Lightweight User Dashboard

The website can include a lightweight user dashboard.

The dashboard should show:

- User basic profile
- Membership status
- Membership expiry date
- Remaining credits or energy points if applicable
- Recent preview records
- App download/open-app CTA
- Upgrade CTA

The dashboard should remain limited and should not replace the app dashboard.

8.8 About / Brand Page

This page should build trust and explain Shuyi's philosophy.

It may include:

- Brand story
- Product mission

Explanation of 数易
Explanation of tarot guidance
Explanation of spiritual/personal growth positioning
App CTA

8.9 FAQ / SEO Pages

Create SEO-friendly pages for:

What is Shuyi
What is 数易
Tarot reading preview
Free 测算
Membership FAQ
App vs website difference
Privacy and data usage
Terms and disclaimers

9. Drag-and-Drop Content Management Requirement

The Shuyi website should support drag-and-drop or visual content editing for selected non-sensitive website sections.

The goal is to allow non-technical admins to update marketing and content areas without depending on developers for every small change.

Drag-and-drop editing should be supported for:

Homepage banners
Hero section text and images
CTA sections
Feature blocks
FAQ sections
SEO content sections
About/brand content
App download banners
Promotional banners
Membership marketing copy
Basic landing page sections

However, drag-and-drop editing should not control core product logic.

The following should remain controlled by frontend code and backend APIs:

- Tarot calculation logic
- Tarot result generation
- 数易 calculation logic
- Membership entitlement rules
- Payment logic
- User records
- Login/authentication
- API response handling
- Security-sensitive pages
- Database-related operations

Important principle:

Marketing content can be drag-and-drop. Product logic should not be drag-and-drop.

This keeps the website easy to update while preserving security, stability, and maintainability.

10. Admin Dashboard / Back Office Requirement

The Shuyi platform should include an admin dashboard for internal team operations.

The admin dashboard should be separate from the public website and should require secure admin login.

The admin dashboard should allow authorized admins to manage users, membership, records, content, tarot configuration, calculation configuration, and operational settings.

10.1 Core Admin Functions

The admin dashboard should support:

- User management
- Membership management
- Payment/package management
- 测算记录 management
- Tarot configuration management
- Calculation type management
- Content/banner management
- FAQ/SEO content management
- App download link management
- Analytics overview
- Support/customer lookup
- Admin role management
- Audit log

10.2 User Management

Admins should be able to:

- Search users
- View user profile
- View registration source
- View membership tier
- View membership expiry date
- View user status
- View recent activity
- View user 测算记录 summary
- Suspend or reactivate user if needed
- Update user remarks or internal notes

Admins should not directly edit sensitive user data unless permission is granted.

10.3 Membership Management

Admins should be able to:

- View user membership tier
- View package purchase history
- View expiry date
- Manually adjust membership if required
- Grant complimentary membership
- Cancel or revoke membership access
- View Elite/VIP entitlement rules
- Manage one-off packages
- Manage app-only benefits vs website-accessible benefits

Membership entitlement logic should still be controlled by backend rules, not manually hardcoded in frontend.

10.4 Tarot / 测算 Management

Admins should be able to:

- Manage available website 测算 types
- Enable or disable calculation types
- Manage tarot lite availability
- Configure preview limits
- Configure result display rules
- Configure app CTA wording
- View website-generated preview records

The admin dashboard should not expose or weaken core calculation logic.

10.5 Content Management

Admins should be able to edit:

- Homepage banners
- Hero text
- CTA wording
- Feature sections
- FAQ content
- SEO title and description
- Membership marketing copy
- Promotional banners
- App download links
- About page content

If drag-and-drop editing is implemented, it should only apply to marketing and content sections, not product logic.

10.6 Role-Based Access Control

The admin dashboard should support different admin roles, such as:

- Super Admin
- Operations Admin
- Content Editor
- Support Admin
- Marketing Admin
- Finance Admin

Each role should only access the functions needed for their work.

10.7 Admin Security Requirements

The admin dashboard must include:

- Secure admin login
- Role-based access control
- Audit log for important actions
- No direct frontend database access
- Backend validation for all admin actions
- Protected membership modification
- Protected user suspension/reactivation

API authentication
Rate limiting where appropriate

11. Tarot Website Development Requirement

The existing Flutter tarot code should be used as reference for:

User flow
Card layout
Animation style
Visual direction
Card selection behavior
Result structure
App-level full experience boundary

However, the website implementation should be rebuilt in React / Next.js.

Do not simply embed Flutter Web unless approved.

The developer should identify:

Which Flutter tarot screens need a website equivalent
Which logic should move to backend API
Which animations should be simplified for web performance
Which results are preview-only
Which features should remain app-only

12. Membership Optimization

The website should support membership conversion without cannibalizing the app.

Recommended membership strategy:

Free website users can try limited results
Logged-in users can view partial records
Elite/VIP users can see their membership status
Full benefits should remain mainly app-based
Website should explain membership clearly
Website should push users to app for deeper value

Membership page should avoid confusion between:

Free preview
Paid report
Elite membership

VIP membership
One-off package
App-only entitlement
Website-accessible entitlement

Backend should provide a membership entitlement API.

Example API:

GET /api/user/membership-status

Response should include:

User ID
Membership tier
Expiry date
Available benefits
Website-accessible benefits
App-only benefits
Upgrade CTA
Deep link to app membership page

13. API Requirements

The website should call backend APIs for all product logic.

Required APIs may include:

Submit simple calculation
Submit tarot lite request
Fetch result summary
Fetch partial calculation history
Fetch membership status
Fetch user profile
Generate app deep link
Save website preview record
Validate input
Track conversion event
Fetch available calculation types
Fetch membership product list
Fetch admin user list
Update admin membership action
Fetch admin dashboard analytics
Manage content blocks

Manage banners
Manage FAQ content

No sensitive business logic should live only in the frontend.

No frontend page should directly query the production database.

14. Design Requirements

The website should feel like an upgraded version of the Shuyi brand.

The design should be:

Mystical
Premium
Clean
Mobile-first
Trustworthy
Smooth
Modern
Consistent with the Shuyi App

The developer should use the current website and Flutter app as visual references.

The design should include:

Light mode
Dark mode
Responsive layout
Reusable cards
Reusable buttons
Tarot card components
Calculation form components
Result summary components
Membership cards
Dashboard cards
Admin dashboard tables
Admin dashboard forms
Loading state
Error state
Empty state
App CTA banner

Dark mode should be intentional, not a simple color inversion.

Suggested dark mode direction:

Deep navy / dark purple background
Soft card surface
Near-white primary text
Soft gray secondary text
Subtle gold or purple accent
Gentle glow effect only where appropriate

15. SEO Requirements

The website must support:

Clean URLs
Page title
Meta description
H1/H2 structure
Open Graph image
Schema markup where appropriate
Fast page loading
Image alt text
Sitemap
Robots.txt
Canonical URL
Mobile performance optimization

SEO landing pages should be created around high-intent keywords such as:

数易测算
免费测算
塔罗占卜
塔罗牌测试
姓名测算
生日测算
个人运势
数易 App

16. Analytics Requirements

Track the following events:

Homepage visit
Calculation page visit
Calculation started

Calculation submitted
Result viewed
Tarot question entered
Tarot card drawn
Tarot result viewed
Membership page viewed
Upgrade CTA clicked
App download clicked
Open app clicked
Login clicked
Record viewed
API error occurred
Dark mode toggled
Admin login
Admin membership update
Admin content update

Analytics should help answer:

Which pages convert users to app downloads
Which calculation types attract the most users
Which tarot flow has the best completion rate
Whether website users later become app users
Whether membership page improves upgrade conversion
Which content sections need improvement

17. Performance Requirements

The website should be optimized for mobile users.

Requirements:

Fast first load
Lazy-load heavy images
Avoid unnecessary JavaScript
Use optimized images
Use CDN caching
Avoid heavy animation on low-end devices
Optimize tarot animation for mobile browser
Use loading skeletons
Avoid blocking API calls where possible
Keep marketing pages lightweight
Keep admin dashboard separate from public website bundle where possible

18. Security Requirements

The website should:

- Use HTTPS
- Never expose secret keys
- Never expose full database data
- Never directly query database from frontend
- Validate all user inputs through backend
- Rate-limit calculation APIs
- Protect membership APIs
- Protect user record APIs
- Use secure authentication/session handling
- Avoid storing sensitive personal data in local storage
- Protect admin routes
- Use role-based access control
- Keep audit logs for sensitive admin actions

19. Migration Plan

- Phase 1: Audit current WordPress / Elementor website and custom code
- Phase 2: Confirm new Next.js information architecture
- Phase 3: Define required APIs from PHP backend
- Phase 4: Confirm drag-and-drop CMS or visual editor approach
- Phase 5: Confirm admin dashboard approach
- Phase 6: Build global design system and dark mode
- Phase 7: Build homepage and SEO pages
- Phase 8: Build 测算 landing page
- Phase 9: Build tarot lite web flow based on Flutter app reference
- Phase 10: Build result page
- Phase 11: Build membership page
- Phase 12: Build lightweight profile/dashboard
- Phase 13: Build partial 测算记录 page
- Phase 14: Build admin dashboard core modules
- Phase 15: Add drag-and-drop content editing for selected sections
- Phase 16: Add analytics, SEO, and performance optimization
- Phase 17: Test mobile, tablet, desktop, Safari, Chrome, and Edge
- Phase 18: Deploy new website
- Phase 19: Redirect old WordPress URLs
- Phase 20: Decommission Elementor/WordPress dependency after verification

20. Acceptance Criteria

The project is complete when:

- Website is no longer dependent on Elementor
- Website is no longer built as WordPress frontend pages
- Frontend is built using Next.js / React / TypeScript
- Existing PHP backend is used through APIs
- Website supports light mode and dark mode
- Website is mobile, tablet, and desktop responsive
- Homepage is completed
- 测算 landing page is completed
- Tarot lite page is completed based on Flutter app reference
- Result preview page is completed
- Membership page is completed
- Lightweight user dashboard/profile page is completed if backend supports it
- Partial 测算记录 page is completed if backend supports it
- Admin dashboard is implemented for core operations
- Admin dashboard supports user management
- Admin dashboard supports membership management
- Admin dashboard supports content/banner/FAQ management
- Drag-and-drop editing is available for selected marketing/content sections
- Drag-and-drop editing does not control product logic
- App download/open-app CTAs are implemented
- SEO basics are implemented
- Analytics events are implemented
- No sensitive logic is placed directly in frontend
- No direct frontend database access exists
- Old WordPress/Elementor pages are audited and redirected
- Website remains intentionally limited compared with the mobile app

21. Developer Deliverables

The developer should provide:

- Current website audit document
- Recommended implementation plan
- Finalized tech stack proposal
- API requirement list
- Updated website design or mockups
- Light mode and dark mode design direction
- Reusable component documentation
- Implemented responsive website pages
- Tarot lite web implementation
- Membership page implementation
- User dashboard implementation if supported

Admin dashboard implementation
Drag-and-drop content editing setup
API integration documentation
List of custom code added or modified
Deployment notes
Admin usage notes
Testing checklist
SEO checklist
Analytics event list
Security checklist

22. Recommended Implementation Priority

Recommended order:

Audit current website and custom code
Confirm design direction
Confirm backend API requirements
Confirm drag-and-drop CMS/page builder approach
Confirm admin dashboard approach
Build design system
Build homepage
Build SEO/content pages
Build 测算 landing page
Build tarot lite page
Build result preview page
Build membership page
Build lightweight user dashboard
Build partial 测算记录 page
Build admin dashboard core features
Add drag-and-drop content editing
Add analytics and SEO
Test and deploy
Redirect old pages
Remove Elementor/WordPress dependency

23. Important Product Principle

The website should attract, educate, preview, and convert.

The app should retain, monetize, and deliver the full experience.

Therefore, the website should give enough value for trust and conversion, but not enough to replace the Shuyi App.

Marketing content can be flexible and drag-and-drop.

Product logic must remain secure, backend-controlled, and stable.

The public website, user dashboard, and admin dashboard should be clearly separated.

24. Final Recommendation

The recommended Shuyi stack is:

Public website: Next.js + React + TypeScript

Styling: Tailwind CSS + reusable component system

Drag-and-drop content: Builder.io, Plasmic, Directus, Strapi, or custom block editor

User dashboard: Next.js + PHP backend API

Admin dashboard: PHP admin panel or React Admin

Backend: Existing PHP backend

Database: Existing MySQL / current database

Tarot web: Rebuild Flutter tarot flow in React / Next.js

Membership: Backend-controlled entitlement APIs

Hosting: Vercel frontend + existing backend server

Analytics: GA4 + ad pixels + backend event tracking

This stack gives Shuyi better long-term control than Elementor and WordPress, while still keeping the Flutter mobile app as the main product and monetization experience.

Appendix: Admin Panel

PRD: Shuyi Website Admin Panel / Back Office

1. Product Name

Shuyi Website Admin Panel / Back Office

2. Product Context

Shuyi is rebuilding its website into a modern, API-driven web frontend using Next.js, React, TypeScript, and reusable UI components.

The website is not intended to replace the Shuyi mobile app. The website should function as:

- A brand and trust-building channel
- An SEO acquisition channel
- A lightweight preview experience
- A conversion funnel into the Shuyi App
- A place to manage public website content, campaigns, SEO pages, banners, and app download CTAs
- A support and operations console for website-related activities

The Shuyi Website Admin Panel is required to manage website content, drag-and-drop sections, SEO, banners, landing pages, campaigns, user support lookup, website-level tarot/测算 configuration, PlenorHub product display integration, analytics, system configuration, integration monitoring, and audit logs.

However, the Website Admin Panel must not duplicate existing admin systems.

3. System Ownership Boundary

The admin architecture must follow a clear ownership boundary.

3.1 KCC ID Responsibility

KCC ID is the source of truth for:

- User identity
- Login

- Authentication
- Password
- Phone/email verification
- Account credentials
- SSO session
- KCC user ID

The Shuyi Website Admin Panel should not manage user passwords, phone verification, email verification, or login credentials.

3.2 Shuyi App Admin Responsibility

The existing Shuyi App Admin Panel is the source of truth for:

- Membership management
- Membership tier
- Membership expiry
- VIP / Elite access
- Energy point / credit balance
- Energy point deduction
- Energy point top-up
- Monthly credit allocation
- Credit rules
- App-level entitlement
- Payment entitlement result
- App-level membership package management

The Shuyi Website Admin Panel should not duplicate these functions.

The Website Admin Panel may read membership, credit, or payment information through backend APIs for display and support purposes, but it must not directly modify them.

3.3 PlenorHub Responsibility

PlenorHub is the source of truth for:

- Marketplace products
- SKU
- Product price
- Product inventory
- Seller management
- Order fulfillment
- Marketplace order status
- Product catalog data

The Shuyi Website Admin Panel should not create or edit PlenorHub products directly.

The Website Admin Panel may control how PlenorHub products are displayed on the Shuyi website.

3.4 Shuyi Website Admin Responsibility

The Shuyi Website Admin Panel is responsible for:

- Website CMS
- Drag-and-drop content editing
- Website page management
- SEO content management
- Homepage banners
- Landing pages
- Campaign pages
- FAQ content
- App download links
- Deep links
- Website tarot/测算 preview configuration
- Website preview record lookup
- PlenorHub product display placement
- Website analytics
- Conversion tracking
- User support lookup
- Payment/credit/membership read-only support view
- System configuration
- Feature flags
- Integration health monitoring
- Admin roles and permissions
- Audit logs

Simple principle:

KCC ID manages identity. App Admin manages membership, payment entitlement, and energy points. PlenorHub manages products. Shuyi Website Admin manages website content, operations, display, support lookup, analytics, configuration, and audit.

4. Product Objective

The objective of the Shuyi Website Admin Panel is to allow the internal team to manage the Shuyi website and website-related operations without requiring developers for every small update.

The admin panel should allow authorized admins to:

- Edit website pages visually
- Manage banners and campaigns
- Update SEO pages
- Manage FAQ and brand content
- Configure app download and deep links
- Control website-level tarot/测算 preview display
- View website-generated preview records
- Select PlenorHub products for display on Shuyi website
- View user information for support purposes
- View membership/payment/credit status in read-only mode
- Track website analytics and conversion
- Monitor KCC ID, PlenorHub, app admin, and backend integration status
- Manage admin roles and permissions
- Review audit logs

The admin panel should support operational independence while preserving system security and source-of-truth boundaries.

5. Core Product Principle

The public website should attract, educate, preview, and convert.

The mobile app should retain, monetize, and deliver the full experience.

The Website Admin Panel should therefore manage the website and operational layer, but not replace the mobile app admin.

Important principles:

1. Marketing content can be drag-and-drop.
2. Product logic must remain backend-controlled.
3. User credentials must remain KCC ID-controlled.
4. Membership and energy points must remain app-admin-controlled.
5. Payment entitlement must remain backend/app-admin-controlled.
6. Product data must remain PlenorHub-controlled.
7. Website admin may show read-only support data, but should not become a second source of truth.

8. All sensitive admin actions must be permission-controlled and audit-logged.
-

6. User Types

6.1 Super Admin

Has full access to all website admin modules.

Responsibilities:

- Manage admin roles
- Configure system settings
- Publish website pages
- Review audit logs
- Control feature flags
- Manage integrations
- Access all support views

6.2 Content Editor

Responsible for website content and CMS editing.

Access:

- Drag-and-drop CMS
- Page manager
- FAQ
- SEO content
- Media library
- Banners
- About page
- Landing pages

No access to system configuration, integration settings, or sensitive user/payment views.

6.3 Marketing Admin

Responsible for campaigns and conversion.

Access:

- Campaign pages
- Banners

- App download CTA
- Deep link settings
- UTM links
- Analytics dashboard
- Product display placement
- Promotional content

No access to user credential, membership modification, credit adjustment, or payment modification.

6.4 Support Admin

Responsible for customer support.

Access:

- User lookup
- Website activity timeline
- Website preview records
- Read-only membership status
- Read-only payment status
- Read-only credit/energy point status if API supports it
- Support notes

No access to modify membership, credits, payment, credentials, or product data.

6.5 Developer Admin

Responsible for technical configuration and monitoring.

Access:

- Integration health
- API logs
- Webhook status
- Feature flags
- Maintenance mode
- Error logs
- Cache clear
- System settings

No access to business-sensitive modifications unless explicitly granted.

6.6 Read-Only Admin

Can view selected dashboards but cannot edit or publish anything.

7. Admin Panel Modules

The Shuyi Website Admin Panel should include the following modules.

Module 1: Admin Dashboard

1.1 Objective

Provide a high-level overview of website performance, content status, conversion, system health, and admin activity.

1.2 Dashboard Sections

The dashboard should show:

- Website visits
- New visitors
- Returning visitors
- Popular pages
- SEO page performance
- App download clicks
- Open-app clicks
- Membership CTA clicks
- Tarot preview starts
- Tarot preview completions
- 测算 preview starts
- 测算 result views
- PlenorHub product clicks
- Active campaigns
- Scheduled banners
- Draft pages
- Published pages
- API error alerts
- KCC ID integration status
- PlenorHub integration status

- App admin API status
- Recent admin activity

1.3 Requirements

The dashboard should:

- Be mobile/tablet/desktop responsive
 - Show date filters
 - Show quick links to key modules
 - Show system alerts clearly
 - Allow export of basic analytics if needed
-

Module 2: Admin Login and Role-Based Access Control

2.1 Objective

Allow secure admin access through KCC ID while maintaining Shuyi-specific admin roles and permissions.

2.2 Requirements

The admin login should:

- Use KCC ID SSO
- Validate admin access through backend
- Map KCC ID user to Shuyi admin role
- Support multiple admin roles
- Support permission-based module access
- Support account suspension/removal for admins
- Log all admin login activity

2.3 Role Management

Super Admin should be able to:

- View admin users

- Assign roles
- Remove roles
- Suspend admin access
- View admin activity
- Review audit logs

2.4 Security

The admin system must include:

- Secure admin login
 - Role-based access control
 - Backend permission validation
 - Admin session expiry
 - Protection against unauthorized route access
 - Audit log for important actions
-

Module 3: Drag-and-Drop Website CMS

3.1 Objective

Allow non-technical admins to visually manage website marketing content without changing code.

3.2 Editable Website Areas

The drag-and-drop CMS should support editing for:

- Homepage
- Hero section
- Feature sections
- CTA sections
- FAQ sections
- About page
- Brand story page
- SEO landing pages
- Membership marketing page
- Tarot landing page marketing sections
- 测算 landing page marketing sections

- App download section
- Promotional banners
- Announcement bar
- Campaign landing pages
- Product recommendation sections
- Footer content where appropriate

3.3 Drag-and-Drop Block Types

The CMS should include reusable blocks:

- Hero block
- Image banner block
- Text block
- Rich text block
- Feature grid block
- CTA block
- App download block
- FAQ accordion block
- Membership comparison display block
- Tarot preview marketing block
- 测算 card block
- Result preview block
- Product recommendation block
- Testimonial block
- Video/image block
- Countdown block
- Announcement block
- Divider/spacer block
- Custom HTML block, if approved and permission-controlled

3.4 CMS Controls

The CMS should support:

- Draft saving
- Preview before publishing
- Desktop preview
- Mobile preview
- Light mode preview
- Dark mode preview
- Publish now
- Scheduled publish

- Unpublish
- Version history
- Rollback
- Approval workflow
- SEO metadata per page
- Media selection
- Link selection
- Internal deep link selection
- External link validation

3.5 Important CMS Boundary

The drag-and-drop CMS must not control:

- Tarot generation logic
- 数易 calculation logic
- Membership entitlement rules
- Energy point rules
- Payment logic
- User records
- Database operations
- Authentication
- API security
- Core backend business logic

The CMS controls content and presentation only.

Module 4: Page, Content, and SEO Management

4.1 Objective

Allow the team to create and manage SEO-friendly website pages.

4.2 Page Manager

Admins should be able to:

- Create page
- Edit page
- Duplicate page
- Delete or archive page
- Publish page
- Schedule page
- Set page visibility
- Assign page template
- Manage page slug
- Manage page category
- Preview page

4.3 SEO Management

Each page should allow configuration of:

- SEO title
- Meta description
- URL slug
- H1
- H2 structure
- Canonical URL
- Open Graph title
- Open Graph description
- Open Graph image
- Robots index/no-index
- Sitemap include/exclude
- Schema markup where applicable
- Image alt text

4.4 Suggested SEO Page Types

The admin should support pages around topics such as:

- 什么是数易
- 免费测算
- 塔罗占卜
- 塔罗牌测试
- 姓名测算
- 生日测算
- 个人运势
- 数易 App
- 数易会员

- App vs Website difference
- Privacy Policy
- Terms of Use
- Disclaimer
- FAQ

4.5 Redirect Manager

The admin should support redirect management for old WordPress/Elementor URLs.

Redirect manager should include:

- Old URL
 - New URL
 - Redirect type
 - Active/inactive status
 - Created by
 - Last updated date
-

Module 5: Banner, Campaign, and Announcement Management

5.1 Objective

Allow admins to manage website promotional content and campaign visibility.

5.2 Banner Management

Admins should be able to manage:

- Homepage banner
- Promotional banner
- App download banner
- Membership campaign banner
- Tarot campaign banner
- 测算 campaign banner
- Product recommendation banner
- Top announcement bar

- Maintenance notice

5.3 Campaign Management

Campaign manager should support:

- Campaign name
- Campaign type
- Start date
- End date
- Campaign landing page
- Banner image
- CTA text
- CTA link
- Deep link
- UTM tracking
- Target page placement
- Active/inactive status
- Campaign analytics

5.4 Announcement Management

Announcements should support:

- Message text
 - Link
 - Display location
 - Start/end time
 - Dismissible setting
 - Target audience if supported
 - Light/dark display preview
-

Module 6: Media Library

6.1 Objective

Provide centralized management for website media assets.

6.2 Requirements

The media library should support:

- Image upload
- Banner upload
- Icon upload
- Video embed link
- File naming
- Folder/category organization
- Alt text
- Preview
- Replace file
- Delete/archive file
- File size optimization
- CDN/storage integration if applicable

6.3 Security

Media upload should include:

- File type validation
 - File size limit
 - Malware/security check where possible
 - Permission control
 - Audit log for file deletion/replacement
-

Module 7: User Lookup and Support View

7.1 Objective

Allow support admins to investigate website-related user issues without modifying sensitive user data.

7.2 User Search

Admins should be able to search by:

- KCC ID

- Email
- Phone number
- Nickname
- User ID
- Registration source if available

7.3 User Profile View

The user profile view should show:

- KCC ID
- Nickname
- Avatar if available
- Registration source
- Account status from KCC ID, read-only
- Website activity
- Recent login source if available
- Recent website preview activity
- Support notes

7.4 Read-Only App Admin Data

If API integration is available, the website admin may display:

- Membership tier, read-only
- Membership expiry, read-only
- Energy point balance, read-only
- Recent energy point deduction summary, read-only
- Payment/package status, read-only
- Entitlement sync status, read-only

7.5 Prohibited Actions

The Website Admin Panel must not allow support admins to:

- Reset password
- Change phone number
- Change email
- Change membership tier
- Extend membership
- Grant VIP
- Revoke VIP
- Add energy points

- Deduct energy points
 - Change credit rules
 - Modify payment status
 - Create manual payment
 - Edit user credential data
-

Module 8: Payment, Membership, and Credit Support View

8.1 Objective

Provide read-only visibility for support and troubleshooting, without duplicating the mobile app admin.

8.2 Payment Support View

Admins may view:

- User payment history
- Transaction status
- Payment channel
- Package purchased
- Transaction reference ID
- Payment time
- Refund status if available
- Entitlement sync status
- Failed payment reason if available

8.3 Membership Support View

Admins may view:

- Membership tier
- Membership start date
- Membership expiry date
- Membership source
- Package type
- Website-accessible benefits

- App-only benefits
- Entitlement status

8.4 Credit / Energy Point Support View

Admins may view:

- Current balance
- Monthly allocation, read-only
- Recent usage history
- Feature that triggered deduction
- Deduction time
- Sync status

8.5 Strict Limitation

This module is read-only.

It must not allow:

- Manual membership grant
- Membership extension
- Membership revocation
- Energy point top-up
- Energy point deduction
- Credit rule editing
- Payment status editing
- Refund processing
- Payment creation

8.6 Source of Truth

The source of truth remains:

- App Admin for membership and energy points
- Payment backend/payment provider for transaction processing
- Backend entitlement system for access rights

The website admin only displays support information from these systems.

Module 9: Membership Page Display Configuration

9.1 Objective

Allow admins to manage how membership is explained and marketed on the website without managing actual membership.

9.2 Editable Items

Admins should be able to edit:

- Membership page copy
- Free / Elite / VIP display text
- Membership comparison table display
- Benefit explanation
- FAQ
- Upgrade CTA wording
- App upgrade deep link
- App download CTA
- Membership campaign banner

9.3 Limitation

This module must not control:

- Actual user membership tier
- Membership expiry
- Membership price
- Membership entitlement logic
- Energy point allocation
- App-level membership benefits
- Payment processing

Membership content on the website is for explanation and conversion only.

Actual membership rules must come from backend/app admin APIs.

Module 10: Tarot and 测算 Website Configuration

10.1 Objective

Allow admins to manage website-level tarot and 测算 preview availability and display rules.

10.2 Tarot Configuration

Admins should be able to configure:

- Enable/disable tarot lite on website
- Tarot landing page display content
- Preview-only disclaimer
- App CTA wording
- Maximum website preview depth
- Save preview option on/off
- Share option on/off
- Maintenance mode for tarot web
- Error fallback message

10.3 测算 Configuration

Admins should be able to configure:

- Enable/disable website 测算 preview
- Enable/disable specific website-level 测算 entry
- Display order of 测算 cards
- Preview-only disclaimer
- App CTA wording
- Result summary display setting
- Save preview option on/off
- Share option on/off
- Maintenance mode for 测算 web
- Error fallback message

10.4 Prohibited Configuration

Admins must not be able to edit:

- Core 数易 calculation formula
- Tarot generation logic
- Backend algorithm
- Membership entitlement bypass
- Payment access control
- User data rules
- Database fields directly

Product logic should remain backend-controlled.

Module 11: Website Preview Record Management

11.1 Objective

Allow support and operations teams to view website-generated tarot/测算 preview records.

11.2 Record List

The admin should show:

- Record ID
- User ID / anonymous session ID
- KCC ID if logged in
- Record type
- Tarot or 测算 category
- Created date
- Result summary
- Preview/full status
- App CTA clicked or not
- Error status if failed

11.3 Record Detail

Record detail should show:

- Input summary, where appropriate
- Result summary

- API status
- Save status
- Share status
- App deep link generated
- User conversion action
- Error log if applicable

11.4 Privacy Limitation

Admins should only see necessary support-level information.

Sensitive personal inputs should be masked or permission-controlled where needed.

Module 12: PlenorHub Product Display Integration

12.1 Objective

Allow Shuyi Website Admin to control how PlenorHub products appear on the Shuyi website without managing product data directly.

12.2 Product Sync View

Admins should be able to view:

- Synced product list
- Product name
- Product image
- Product category
- Product price, read-only
- Product availability, read-only
- PlenorHub product ID
- Product deep link
- Sync status
- Last sync time

12.3 Display Management

Admins should be able to:

- Select featured products
- Place products into CMS product blocks
- Choose display order
- Hide product from Shuyi website display
- Add Shuyi-side display label
- Configure CTA wording
- Configure product placement by page
- Configure campaign-product mapping

12.4 Product Click Analytics

Admin should track:

- Product impressions
- Product clicks
- Click source page
- Campaign source
- Deep link clicks
- PlenorHub referral clicks

12.5 Limitation

The Website Admin Panel must not manage:

- SKU creation
- Product price
- Product inventory
- Seller information
- Order fulfillment
- Marketplace order processing
- Product approval workflow inside PlenorHub

PlenorHub remains the source of truth for product data.

Module 13: App Download and Deep Link Management

13.1 Objective

Allow admins to manage app download links and app conversion paths.

13.2 App Download Settings

Admins should be able to manage:

- iOS App Store link
- Android Play Store link
- Android APK fallback link if needed
- QR code
- App download banner
- App open fallback behavior

13.3 Deep Link Settings

Admins should be able to manage:

- Open app homepage
- Open app membership page
- Open app tarot page
- Open full tarot result in app
- Open full 测算 report in app
- Open user profile page in app
- Open PlenorHub product page if applicable

13.4 Tracking

Each CTA/deep link should support:

- UTM source
 - Campaign name
 - Page source
 - Click count
 - Conversion event
 - Error fallback if app is not installed
-

Module 14: Analytics and Conversion Dashboard

14.1 Objective

Allow internal team to understand how the website performs as an acquisition and conversion channel.

14.2 Events to Track

Track:

- Homepage visit
- SEO page visit
- Tarot page visit
- Tarot question entered
- Tarot card drawn
- Tarot result viewed
- 测算 page visit
- 测算 started
- 测算 submitted
- Result viewed
- Membership page viewed
- Upgrade CTA clicked
- App download clicked
- Open app clicked
- Login clicked
- KCC ID login success
- KCC ID login failed
- PlenorHub product viewed
- PlenorHub product clicked
- FAQ expanded
- Campaign banner clicked
- API error
- Dark mode toggled
- Form submission error

14.3 Dashboard Filters

Analytics should support filtering by:

- Date range
- Page
- Source
- Campaign
- Device type
- Logged-in vs anonymous
- New vs returning user
- Tarot vs 测算
- CTA type

14.4 Key Questions Analytics Should Answer

The dashboard should help answer:

- Which pages attract users?
 - Which SEO pages bring traffic?
 - Which tarot/测算 flow converts best?
 - Which CTA drives app downloads?
 - Which campaign generates the most app opens?
 - Which product block gets clicks?
 - Where do users drop off?
 - Which API errors affect conversion?
 - Whether users move from website preview to app usage
-

Module 15: Support Console

15.1 Objective

Provide support team with a unified place to check website-related user issues.

15.2 Support Functions

Support admins should be able to:

- Search user
- View website activity timeline
- View preview records
- View login status indicator
- View KCC ID link status

- View read-only membership status
- View read-only payment status
- View read-only credit/energy status
- View PlenorHub referral activity
- View API error logs related to user
- Add internal support notes
- Tag issue type
- Export support summary

15.3 Issue Tags

Support issue tags may include:

- Login issue
- Membership display issue
- Payment display issue
- Energy point inquiry
- Tarot result issue
- 测算 result issue
- App download issue
- Deep link issue
- Product link issue
- Website bug
- API error

15.4 Limitation

Support console is for investigation and notes only.

It should not modify source-of-truth data from KCC ID, App Admin, payment system, or PlenorHub.

Module 16: System Configuration and Feature Flags

16.1 Objective

Allow authorized admins/developers to control website configuration safely.

16.2 Feature Flags

Feature flags should include:

- Enable/disable tarot lite website
- Enable/disable 测算 preview website
- Enable/disable login on website
- Enable/disable membership page CTA
- Enable/disable PlenorHub product blocks
- Enable/disable app deep links
- Enable/disable campaign banner
- Enable/disable website maintenance mode

16.3 System Settings

Settings may include:

- API endpoint configuration
- Public website maintenance message
- Error fallback message
- App download fallback
- Rate-limit setting
- Cache clear
- CDN/media status
- Environment indicator
- Webhook endpoint status

16.4 Permission

Only Super Admin or Developer Admin should access this module.

All changes must be audit-logged.

Module 17: Integration Health Monitor

17.1 Objective

Monitor the health of integrations between the Shuyi website and external/internal systems.

17.2 Integrations to Monitor

The admin should show status for:

- KCC ID login
- KCC ID user profile API
- App Admin membership read-only API
- App Admin credit read-only API
- Payment read-only API
- PlenorHub product sync API
- PlenorHub deep link API
- Backend tarot/测算 API
- Analytics events
- Media/CDN service
- Email/SMS if used for website notifications

17.3 Monitoring Information

Show:

- Integration name
 - Current status
 - Last successful response
 - Last failed response
 - Error message
 - Response time
 - Last sync time
 - Retry status
 - Webhook delivery status if applicable
-

Module 18: Audit Log and Admin Activity Log

18.1 Objective

Track important admin actions for security, accountability, and debugging.

18.2 Actions to Log

The audit log should record:

- Admin login
- Admin logout
- Failed admin login
- Role changed
- Admin access removed
- Page created
- Page edited
- Page published
- Page unpublished
- SEO changed
- Banner changed
- Campaign created
- Campaign changed
- Media uploaded
- Media deleted
- CTA link changed
- Deep link changed
- Product placement changed
- Tarot/测算 setting changed
- Feature flag changed
- Maintenance mode changed
- Support note added
- Integration setting changed
- Cache cleared
- Redirect changed

18.3 Audit Log Fields

Each log should include:

- Admin name
- Admin KCC ID
- Admin role
- Action type
- Module
- Target item
- Before value
- After value
- Reason, where required
- Timestamp
- IP address if available
- Device/browser if available

18.4 Read Access

Only Super Admin and authorized Developer Admin should view full audit logs.

8. Data Ownership Matrix

Data / Function	Source of Truth	Website Admin Permission
User login	KCC ID	No edit
Password	KCC ID	No access
Phone/email verification	KCC ID	No edit
User basic profile	KCC ID / backend	Read-only
Membership tier	App Admin	Read-only
Membership expiry	App Admin	Read-only
Membership entitlement	App Admin / backend	Read-only
Energy point balance	App Admin	Read-only
Energy point rules	App Admin	No edit
Energy point top-up/deduction	App Admin	No edit
Payment transaction	Payment backend / App Admin	Read-only
Refund processing	Payment backend / App Admin	No edit
Product SKU	PlenorHub	No edit
Product price	PlenorHub	Read-only
Product inventory	PlenorHub	Read-only
Product display placement	Shuyi Website Admin	Editable
Website content	Shuyi Website Admin	Editable
Website banners	Shuyi Website Admin	Editable

Website SEO pages	Shuyi Website Admin	Editable
Website deep links	Shuyi Website Admin	Editable
Website tarot/测算 display config	Shuyi Website Admin	Editable
Core calculation logic	Backend	No edit
Tarot generation logic	Backend	No edit
Admin roles	Shuyi Website Admin	Editable by Super Admin
Audit logs	Shuyi Website Admin	Read-only after creation

9. API Requirements

The Website Admin Panel should communicate with backend APIs.

No admin frontend should directly query the production database.

Required API categories:

9.1 Admin Authentication API

- Admin login through KCC ID
- Admin role validation
- Admin permission check
- Admin session check

9.2 CMS API

- Create page
- Edit page
- Publish page
- Schedule page
- Save draft
- Retrieve page version
- Rollback page
- Manage content blocks
- Manage banners
- Manage FAQ
- Manage media

9.3 SEO API

- Update SEO metadata
- Manage slug
- Manage canonical URL
- Manage sitemap status
- Manage robots status
- Manage redirects

9.4 User Support API

- Search user
- Fetch user website activity
- Fetch preview records
- Fetch read-only membership status
- Fetch read-only energy point status
- Fetch read-only payment status
- Add internal support note

9.5 Tarot / 测算 Website Config API

- Fetch available website calculation types
- Enable/disable website calculation type
- Enable/disable tarot lite
- Update preview display settings
- Update CTA wording
- Update disclaimer
- Update maintenance message

9.6 PlenorHub Integration API

- Fetch synced product list
- Fetch product details, read-only
- Save website product placement
- Hide product from website display
- Fetch product sync status
- Fetch product click analytics

9.7 Deep Link API

- Manage app links
- Generate deep links
- Validate deep links
- Track deep link clicks

9.8 Analytics API

- Fetch dashboard metrics
- Fetch conversion metrics
- Fetch campaign performance
- Fetch page performance
- Fetch error rate
- Export analytics report

9.9 System API

- Fetch integration health
- Fetch API error logs
- Clear cache
- Update feature flags
- Update maintenance mode
- View webhook status

9.10 Audit Log API

- Create audit log
 - Fetch audit logs
 - Filter audit logs
 - Export audit logs if permitted
-

10. Technical Requirements

10.1 Recommended Admin Frontend

The admin panel can be built using one of the following approaches:

Option A:

- React Admin / Refine / Ant Design Pro
- API-driven admin dashboard
- Good for faster development

Option B:

- Custom Next.js admin dashboard
- Full control over UI and permission structure
- Better if Shuyi wants a highly customized admin experience

Option C:

- PHP-based admin dashboard
- Suitable if backend developer is stronger in PHP
- Faster integration with existing PHP backend

Recommended practical approach:

- Public website: Next.js / React / TypeScript
- Website admin: React-based admin dashboard or PHP-based admin depending on developer strength
- Backend: Existing PHP backend as API/business logic layer
- Database: Existing database, accessed only through backend APIs
- CMS: Custom block-based editor, Builder.io, Plasmic, Directus, Strapi, or similar
- Authentication: KCC ID admin login
- Analytics: GA4 plus backend event tracking

10.2 Backend Control

The backend must control:

- Permission validation
- User data access
- Membership read-only API
- Payment read-only API
- Energy point read-only API
- Tarot/测算 APIs
- Product sync logic
- Audit log creation
- Security validation

10.3 No Direct Database Access

The admin frontend must not directly access the database.

All admin actions must go through backend APIs.

11. Security Requirements

The Website Admin Panel must include:

- HTTPS

- Secure admin login
- KCC ID SSO
- Role-based access control
- Permission validation on backend
- No direct frontend database access
- No secret keys exposed in frontend
- No sensitive data stored in local storage
- Input validation
- Rate limiting where appropriate
- Admin session timeout
- Audit logs
- Restricted access to support data
- Restricted access to system settings
- Protected publish actions
- Protected feature flag changes
- Protected integration settings

Sensitive support views should be permission-controlled.

12. UX / Design Requirements

The admin panel should be:

- Clean
- Professional
- Easy for non-technical staff
- Fast to load
- Mobile/tablet responsive where practical
- Optimized for desktop admin use
- Clear in permissions
- Clear in warning messages
- Clear in draft/publish status
- Clear in read-only vs editable modules

The admin UI should include:

- Tables
- Filters
- Search
- Date range picker
- Status badges
- Empty states

- Loading states
- Error states
- Confirmation modals
- Preview panels
- Mobile/desktop preview
- Audit log details
- Save/publish confirmation

Read-only modules should be clearly labeled as read-only.

13. Key Out-of-Scope Items

The Website Admin Panel should not include:

- User password reset
- User credential editing
- Phone/email verification editing
- Membership tier adjustment
- Membership expiry adjustment
- VIP/Elite grant
- VIP/Elite revoke
- Energy point top-up
- Energy point deduction
- Credit rule management
- Monthly credit allocation
- Payment creation
- Payment status editing
- Refund processing
- Product SKU creation
- Product price editing
- Product inventory editing
- Seller management
- PlenorHub order fulfillment
- Core tarot logic editing
- Core 数易 formula editing
- Direct database editor

These functions must remain in their respective source-of-truth systems.

14. Acceptance Criteria

The admin panel is considered complete when:

1. Admin login works through KCC ID.
2. Admin roles and permissions are implemented.
3. Super Admin can manage admin access.
4. Drag-and-drop CMS is available for selected marketing/content sections.
5. Content editors can create, edit, preview, publish, schedule, and rollback pages.
6. SEO metadata can be configured per page.
7. Homepage banners and promotional banners can be managed.
8. FAQ and About content can be managed.
9. App download links and deep links can be managed.
10. Membership page display content can be edited without changing actual membership entitlement.
11. Website tarot/测算 preview configuration can be managed without touching backend logic.
12. Website preview records can be searched and viewed.
13. User support lookup is available.
14. Membership status is shown as read-only if app admin API supports it.
15. Payment status is shown as read-only if payment/app admin API supports it.
16. Energy point status is shown as read-only if app admin API supports it.
17. PlenorHub synced products can be viewed.
18. Admin can select PlenorHub products for website display.
19. Admin cannot edit PlenorHub product data directly.
20. Analytics dashboard tracks website conversion events.
21. Integration health monitor shows KCC ID, PlenorHub, backend, app admin API, and analytics status.
22. Feature flags and maintenance mode are available to authorized admins.
23. Audit log tracks all important admin actions.
24. Admin frontend does not directly query the database.
25. No membership, credit, or payment modification is available in Website Admin.
26. No user credential management is available in Website Admin.
27. All sensitive actions are backend-validated.
28. The admin UI clearly distinguishes editable modules from read-only support views.

15. Developer Deliverables

The developer should deliver:

- Final admin information architecture
- Permission matrix
- Admin dashboard UI
- Admin login integration with KCC ID

- Role-based access control
- Drag-and-drop CMS implementation
- Page/content manager
- SEO manager
- Banner/campaign manager
- Media library
- Membership display configuration
- App download/deep link manager
- Tarot/测算 website configuration
- Website preview record management
- User support lookup
- Payment/membership/credit read-only support view
- PlenorHub product display integration
- Analytics dashboard
- Integration health monitor
- System configuration / feature flags
- Audit log
- API integration documentation
- Security checklist
- Testing checklist
- Admin user guide
- Deployment notes